

컴퓨터 프로그래밍 3 - Term Project 보고서

202302599 이정찬

1. 프로젝트 개요

해당 프로젝트는 컴퓨터 프로그래밍 3 Term Project에 대한 것으로, Mini Student Shell, 이른바, Mini Shell을 구축하는 것이 목적으로,

주어진 PDF 양식에 맞춰 구조를 축조하여, 주어진 각 케이스를 알맞게 처리함에 목적을 둔다.

해당 Shell은 Admin / Client 두 가지 Flag를 설정하여 실행할 수 있으며, 실행 명령어는 Makefile을 통해 관리한다.

해당 Shell은 기본적으로 “학생”의 정보를 관리하는것에 목적을 두며 실행시 설정한 flag에 따라 학생 정보 관리에 있어 사용할 수 있는 명령어에 차이가 있다.

main.c 에서는 사용자의 입력을 받고, 해당 입력에 맞는 함수를 호출하여 사용자의 요청을 처리한다.

command.c 에서는 사용자가 입력한 명령어를 파악하여 명령어 실행 권한을 확인 후에 이를 실제 명령어가 들어있는 student.c 혹은 file_io.c를 호출하여 명령어를 처리한다.

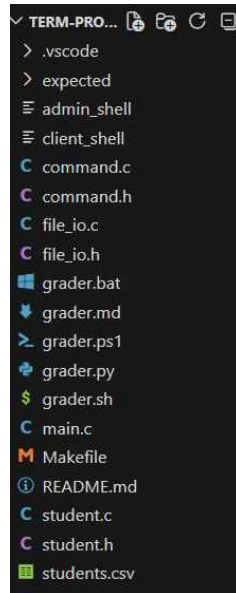
file_io.c 에서는 csv 파일 혹은 명령어 파일 등, 파일 입출력에 대한 요청을 실질적으로 처리한다.

student.c 에서는 사용자가 입력한 명령어를 실질적으로 처리한다.

2. 프로그램 구조

project/

- |—— main.c
- |—— student.h
- |—— student.c
- |—— file_io.h
- |—— file_io.c
- |—— command.h
- |—— command.c
- |—— Makefile
- |—— README.md
- |—— students.csv



PDF에서 제공하는 권장 파일 형식을 그대로 따른다.

정확한 파일 구조는 위 사진과 같다.

Makefile

- 사용자의 명령어, 이를테면 make 혹은 make admin을 받아 이에 맞게 flag를 세워 컴파일을 실행하는 파일로, 일반적으로 사용하는 Makefile과 동일하다.

```
CC      = gcc
CFLAGS = -Wall -Wextra -std=c11

SRCS    = main.c student.c command.c file_io.c
.PHONY: all admin client clean
all: admin client
admin:
    $(CC) $(CFLAGS) -DADMIN_MODE $(SRCS) -o admin_shell
client:
    $(CC) $(CFLAGS) -DCCLIENT_MODE $(SRCS) -o client_shell
clean:
    rm -f admin_shell client_shell *.o
```

정확한 형태는 위와 같으며, default가 all임으로, make만 명령어로 전달할시 admin과 client 모두 재컴파일한다.

이 밖에 clean을 통해 컴파일 결과물을 삭제할 수 있다.

README.md

- 사용 명세에 대해 간략하게 적어놓은 부분으로, 해당 부분을 참고하여 Mini Shell의 작동 방식과 설계 방식에 대해 이해할 수 있다.
- 보고서의 축약 버전이라 보면 된다.

student.csv

- 학생 정보를 저장하는 csv 파일로, id,name,score 순으로 각 column이 존재하며, 이후 각각의 정보가 하나의 row가 되어 csv에 하나의 데이터로써 저장될 것이다.

main.c

- 사용자의 입력을 받아 명령어 처리로 분기하는 역할을 한다.
- 초기 입력에 따라 커맨드 텍스트 파일을 읽어오거나, 혹은 Mini Shell로 바로 진입한다.
- 커맨드 텍스트 파일을 입력으로 받은 경우엔, 해당 텍스트 파일 실행 후, 즉시 Shell을 종료한다.
- 추가적으로 다른 소스파일로 사용자에게 입력받은 csv의 경로(이름)과 함께 reload, 즉 파일을 처음 읽어오는 것이 다시 읽어오는것인지에 대한 상태를 나타낼 변수도 같이 넘긴다.
- 또한, 학생 정보를 가져올 때 링크드 리스트로 관리하기에 head가 필요한데, 이때의 head를 main.c에서 전달하여준다.
- 또한, exit 명령어의 경우는 main.c에서 처리한다.
- 사용자의 입력을 command.c에 정의된 **cmd_process** 명령어를 통해 처리한다.

student.h

- PDF에 주어진 구조체에 대한 정의(다른 소스 코드에서도 활용할 여지가 있기 때문에 헤더에서 정의)와 함께 student.c 함수들의 선언이 되어있다.
- 중복 정의를 막기 위해 #pragma once를 활용한다.

student.c

- Mini Shell에 진입한 후 실행할 명령어의 Body가 정의되어 있는 부분으로, add ,create ,update ,delete ,stats ,list ,find 등의 명령어가 존재하며,
- 내부적으로 사용할 create함수(학생 정보를 Linked List로 관리하라는 조건

이 있었기에 Node 생성 함수가 필요함.)와 해당 노드들을 전부 해제해줄 free_student(메모리 해제 역할) 함수가 추가로 존재한다.

- 구조체 정의가 헤더에 있으므로 헤더 또한 include한다.

file_io.h

- 헤더파일임으로 혹시모를 충돌에 대비해 #pragma once 처리를 해놓았으며, 소스 코드 내에서 활용해야할 구조체 내용을 전달해주기 위해 student.h 헤더를 include한다.

file_io.c

- 파일 입출력에 대한 명령어 및 요청을 처리한다.
- main.c를 통해 처음 프로그램이 빌드되면, csv 파일을 읽어올 수요가 있다. 이때 main.c에서 넘겨준 is_reload란 flag 역할의 변수와 함께 읽어온 파일 경로(이름)인 csv_path를 받아 해당 파일을 불러온다.
- 이때 파일을 불러오는 역할은 reload 함수가 담당하며, 현재까지의 작업 목록을 불러온 csv 파일에 저장하는 역할은 save 함수가 담당한다.
- save와 reload 함수, 두 함수의 정의만을 품고 있다.

command.h

- 사용자의 명령어와 해당 명령어가 정의된 소스코드를 이어주는 일종의 hub 역할을 수행함으로, student.h 와 file_io.h 헤더 파일을 모두 include하며 혹시 몰라 해당 헤더도 #pragma once를 적용하였다.
- main.c에서 직접적으로 호출하여 사용할 함수로 cmd_process 단 하나만을 외부적으로 노출한다.

command.c

- 각 명령어에 대한 정의가 끝났으니 사용자의 입력과 해당 명령어를 연결해 줄 부분이 필요하다.
- 그러한 부분을 command.c에서 처리를 해준다.
- 명령어에 PDF에서 명시된 각 에러 케이스에 대한 처리와, 명령어 성공 여부, CLIENT / ADMIN flag별 실행가능 명령어 분기, 사용자 명령어와 해당 프로젝트 소스 코드에 정의된 명령어 정의와의 연결을 직접적으로 수행하는 부분이다.
- 매크로를 통해 각 flag 선언 여부에 따라 사용가능한 명령어 테이블을 분기

하며, 이는 PDF에 주어져있었으나, 어느정도 테이블을 구성한 후에 확인해버리는 바람에, 이미 만들어놓은 것을 그냥 그대로 사용하였다.

- PDF의 enum을 활용하면 훨씬 더 깔끔하게 에러를 확인하는 로직을 짤 수 있었을터이지만, 이미 만들어놓은 터라, 각 에러 케이스별 오류 체크를 위해 `add_check` / `is_digit` / `update_check` 등 제대로 했다면 다른 형태로 존재했을 에러 확인용 함수들이 존재한다.
- `help` 명령어 처리시에 제공할 테이블 위해 새롭게 정의된 구조체가 하나 더 존재하며 이 또한 기존 PDF에 제공된 양식과는 다른 부분이지만, 결과적으로는 같은 바를 추구한다.

3. 자료구조 설계

각 명령어를 통해 만들어진 “학생 정보”는 Singly Linked List 형태이며, 만들어진 순서대로 꼬리에 가깝게 배치가 된다.

프로그램 처음 시작시, `main.c`에서 `reload` 함수를 호출하며, 이때 `main.c`에서 `reload` 함수의 인자로 `head`를 `NULL`로 넘겨주면, `reload` 함수 내부적으로 `add` 함수를 호출하는 과정에서 `head`가 `NULL`일 때 삽입된 원소가 되며, 이후 원소는 꼬리 쪽에 계속해서 붙게 된다.

`command.c` 내부에 존재하는 매핑 함수는 오류 탐지를 하여 실패/성공 여부를 파악하기 위해 `int` 자료형을 `return`하게 되어있다. `student.c` 내부에 존재하는 실제 명령어 정의 부분은 `void`를 반환하며, `file_io.c`에 존재하는 `save, reload` 함수의 경우는 읽거나, 저장한 데이터의 수를 나타내기 위해 `int`형을 반환하도록 되어있다.

4. CSV 파일 형식

`id,name,score` 순으로 `column`의 속성이 존재하며, 각 데이터는 행 단위로 존재하며, 하나의 행당 한 명의 학생 정보가 존재한다.

5. 명령어 명세

빌드 명령어

make (all)

- admin과 client 모드의 shell 둘 모두를 새롭게 빌드한다. 다만 수정이 없을 시 새롭게 빌드됨을 보장할 수 없다. (시간 테이블 비교를 통해 새롭게 빌드하는 매니저 특성 상).
- 뒤에 admin 혹은 client를 추가해 원하는 모드의 shell만 빌드할 수 있다.

clean

- 컴파일된 빌드 파일을 모두 삭제한다.

미니 셸 진입 명령어

./admin_shell(or client_shell) (파일명(students.csv)) (-f 커맨드파일명)

- 모드별로 해당하는 Flag가 선언된, 즉 Admin/Client 둘 중 하나의 모드로 Mini Shell을 실행하겠다고 각 모드별로 PDF에 명시된 대로 사용 가능한 명령어가 다름. 이는 아래에서 구체적으로 명세.
- 커맨드 파일을 달아주면 해당 커맨드 파일을 읽어와 내부에 있는 명령어 실행 후 Mini Shell을 즉시 종료

미니 셸 명령어

ADMIN 전용

save `int save(Student * head, const char * filename)`

- file_io.c에 정의되어 있는 함수로, 현재 추가하거나, 수정하거나, 삭제한 학생 정보, 즉 작업한 내용을 바탕으로 현재 파일을 수정하는 명령어.
- fprintf를 통해 쓰기를 수행하며 이때 모드는 w, write를 사용한다.
- head 포인터를 넘겨받은 후 curr 포인터가 복사받아, 한칸씩 나아가며, 처음 추가한 순서대로 파일에 써넣어가며, count를 매삽입하다 하여 결과적으로 성공적으로 써넣은 학생 정보의 수를 int형으로 반환한다.
- 이때 발생할 수 있는 오류는 파일 포인터를 불러오지 못했을 경우이다. 해당 케이스에선 Error: save fail.을 띄우며 에러 flag들이 전달되어 사용자가 최종적으로 save가 실패했음을 알 수 있게 한다.

add <id> <name> <score>

```
void add(Student **head, int id, char * name, int score)
```

- 학생 정보를 추가하는 메서드로, 가장 많은 에러케이스를 가지고 있는 메서드.
- student.c에 정의되어 있으며, student.c에서는 command.c에서 에러 검사를 해주었기에 바로 삽입만을 진행.
- 내부적으로 create 함수에 인자를 넘겨 새롭게 학생 정보에 해당하는 구조체 포인터를 만들어 낸후, 이를 학생 정보 Singly Linked List 꼬리에 달아주려 시도하는데, 이때 만약 head가 NULL인 경우는 방금 삽입하려 한 노드를 head로 만든 후에 종료한다. 이때 head의 값을 수정하기 위해 **head로 받는 것이다.
- command.c에서는 처음 add 명령어와 함께 온 인자들의 유효성을 검사하는데, 해당 명령어 확인 중에 사용자가 보내온 인자들이 훼손되면 안되기에 temp 배열에 해당 명령어 문자열을 복사하여 검사하며, add_check라고 따로 정의해놓은 함수를 통해 **빠진 인자가 존재하는지, 숫자여야 하는 인자가 숫자가 정말 맞는지, 유효 범위를 넘어가는 값으로 들어왔는지를 판단하며, 위에 해당하지 않는다면 해당 명령어를 타입에 맞게 쪼갬 후에 add 함수를 호출하기 전에 마지막으로 중복으로 존재하는 ID인지 find함수를 활용해 확인 후에, add 함수를 호출하여 삽입을 완료한다.**

delete <id>

```
void delete(Student ** head, int id)
```

- 해당하는 id의 학생 정보를 제거하는 메서드로, 똑같이 command.c에서 에러 체크, student.c에서 삭제 로직 수행으로 구성돼있다.
- delete 메서드의 경우는 추가적인 에러 확인 조건이 없었기에 임의로 구성하였다.
- id 인자가 없는 경우, id가 숫자가 아닌 경우, 해당하는 id가 없는 경우 위 3가지 경우를 체크 한 후, student.c에 있는 delete 메서드를 호출하여 실제 해당 id 학생 정보 삭제를 처리하며,
- 이때 하나 남아있던 정보를 삭제하는 경우 head를 NULL로 바꿔야할 소요가 있어 ** head 형태로 넘긴다. (사실 통일성 때문에 head 바꿀 필요 없는 메서드들도 다 이중 포인터 형태이긴 하다.)
- Singly Linked List 삭제 원리처럼, 삭제하고자 하는 노드 이전의 다음 노드 래퍼런스를 다음 노드의 다음 노드로 이어준후, 삭제하려던 노드는 free시킨다.

update <id> <score> `void update(Student ** head, int id, int score)`

- 해당하는 id의 학생의 score 정보를 넘겨받은 score로 업데이트 하는 메서드로 위에서 말한 이중 포인터 헤드가 쓸 필요 없지만, 통일성 때문에 그냥 이중 포인터로 head를 받은 함수로, 이전과 동일하게 에러는 command.c 처리는 student.c에서 진행한다.
- 단순히 해당 id에 해당하는 학생 score만 바꾸기에 로직은 설명할 부분이 거의 없으며, 에러 처리의 경우는 PDF 조건에 맞게 인자 부족, 숫자 인자의 숫자가 아닌 형태, 유효 범위를 넘어서는 인자 세가지 경우와 더불어 없는 ID 업데이트 시도 의 4가지 에러를 command.c에서 확인하며, add처럼 따로 에러 체크용 함수를 두어 이를 확인한다.

ADMIN / CLIENT 모두 사용 가능

find <id> `Student * find(Student ** head, int id)`

- 해당하는 id의 학생 정보를 찾는 메서드로, 앞전과 동일하게 command.c에서 에러 확인 / student.c에서 로직 처리이며, 이중 포인터의 사용 이유는 앞서 기술하였다.
- atoi를 통해 id를 정수형으로 변환하나, 이때 에러 탐지는 하지 않는다.
- 해당 ID를 찾지 못한 경우는 Error를 띄운다. 이때의 출력 포맷은 PDF의 명세에 따른다.

reload `int reload(Student **head, const char * filename)`

- 파일을 다시 불러오거나, 초기에 입력받은 파일을 여는 로직을 담당하며 main.c에서 넘겨준 is_reload가 1이면 다시 불러오기, 0이면 초기 파일 로딩으로 판단하여 이에 맞는 포맷을 출력한다.
- csv를 읽어올거라 가정하고 header를 확인하여 헤더의 유효성을 검사하며 이때 문제가 있을 시 PDF 명세에 따라 에러를 출력한다.
- 파일 불러오기에도 실패한 경우 에러를 출력한다..
- reload 함수는 어떠한 경우에도 head를 넘겨 free_student, 즉 동적 할당된 모든 학생 노드들을 해제한 후에 파일을 읽어 새롭게 데이터를 가져온다.

list `void list(Student ** head)`

- 앞선 메서드들과 동일하게 체크후 해당 함수를 수행한다.
- head가 NULL인 즉, empty List의 경우에는 No students found라는 Error 문구를 띄운다.
- 해당 경우가 아닌 경우, head를 복사한 curr 포인터를 통해 리스트를 순회하며 학생 데이터를 포맷에 맞게 출력한다.

stats `void stats(Student ** head)`

- list와 비슷한 경우의 수로 동작한다. 빈 리스트의 경우는 No student data available Error 문구를 띄우고, 그렇지 않다면 student.c에 위치한 함수를 호출해 PDF 명세에 맞는 포맷을 출력한다.

clear `int cmd_clear(Student ** head, char * args)`

- 터미널을 비우는 명령어로 해당 명령어는 command.c에서 바로 처리하며, printf("\033[2J\033[H"); 출력 후 바로 반환한다.
- 기존에는 student.c 혹은 file_io.c에 존재하는 함수의 인자와 반환 부분을 복사하여 올렸지만, clear 명령어부터는 로직을 분리할만큼 크지 않아 command.c의 매핑 함수를 첨부

exit

- 유일하게 main.c에서 직접처리하는 명령어로, 사용자의 입력이 exit과 동일하다면, free_student(&head)를 통해 현재 불러온 데이터를 모두 해제한 후 Goodbye,를 출력한 후 Mini Shell을 종료한다.

help `int cmd_help(Student ** head, char * args)`

- 명령어 별로 매핑해놓은 구조체 배열을 순회하며 각 명령어에 해당하는 명세를 출력하는 명령어
- 보기 좋게 각 출력 포맷을 %-30s %-30s 로 맞추었다.(각각 함수 이름, 함수 명세)
- 구조체 배열에서는 해당 함수를 필요로 하고, 해당 함수는 구조체 배열을 필요로 하는 상태라, 구조체 배열 전에 해당 함수 선언만 하고, 구조체 배열 선언 후에 함수를 정의하는 형태를 띄도록 하였다.

6. Admin/Client Program 설계

Makefile을 확인하여보면,

```
admin:
    $(CC) $(CFLAGS) -DADMIN_MODE $(SRCS) -o admin_shell
client:
    $(CC) $(CFLAGS) -DCLIENT_MODE $(SRCS) -o client_shell
```

위와 같은 부분을 통해 Flag를 선언함을 알 수 있다.

각 모드별로 출력 포맷과 사용가능 명령어가 조금씩 상이한데,

```
#ifndef ADMIN_MODE
    #define MODE "admin> "
#elif defined(CLIENT_MODE)
    #define MODE "client> "
#else
    #error "Define either -DADMIN_MODE or -DCLIENT_MODE when compiling."
#endif
```

위는 maic.c 소스코드 부분으로, 최상단에 각 flag 정의 별로 Mini Shell에 나타날 모드를 분기시켰으며, 위와 같은 방식으로, 초기 실행시 배너 또한 분기시켰다.

```
#ifndef ADMIN_MODE
printf("[Admin Program]\n");
#elif defined CLIENT_MODE
printf("[Client Program]\n");
#endif
```

아래는 command.c 내부에서 각 모드별로 사용 가능한 명령어와 명령어에 대한 명세를 품고 있는 구조체 배열이 서로 상이한 것을 나타내는 코드이다.

```
#ifndef ADMIN_MODE
CMD table[] = {
    {"add",cmd_add},
    {"delete",cmd_delete},
    {"update",cmd_update},
    {"find",cmd_find},
```

```

        {"save",cmd_save},
        {"reload",cmd_reload},
        {"list",cmd_list},
        {"stats",cmd_stats},
        {"clear",cmd_clear},
        {"help",cmd_help}
    };

    HELP h_table[] = {
        {"save", "Save students to CSV"},
        {"reload", "Reload students from CSV"},
        {"add <id> <name> <score>", "Add a student"},
        {"delete <id>", "Delete a student"},
        {"update <id> <score>", "Update student score"},
        {"find <id>", "Find student by ID"},
        {"list", "List all students"},
        {"stats", "Show statistics"},
        {"clear", "Clear screen"},
        {"exit", "Exit program"},
        {"help","Show help"}
    };

    #elif defined CLIENT_MODE
    CMD table[] = {
        {"find",cmd_find},
        {"reload",cmd_reload},
        {"list",cmd_list},
        {"stats",cmd_stats},
        {"clear",cmd_clear},
        {"help",cmd_help}
    };

    HELP h_table[] = {
        {"reload", "Reload students from CSV"},
        {"find <id>", "Find student by ID"},
        {"list", "List all students"},
        {"stats", "Show statistics"},
        {"clear", "Clear screen"},
        {"exit", "Exit program"},
        {"help","Show help"}
    };

    #endif

```

위를 통해 명령어 명세에서 설계한 대로 각 Mode 별로 사용가능한 명령어가 상이함을 확인할 수 있다.

cmd_process 명령어를 통해 사용자의 명령어가 각 명령어의 매핑 함수로 찾아가는데, 이때 위에서 설계한 구조체 배열을 순회하며 순회 실패시 Unknown command or permission denied 문구를 띄우도록 설계하였다.

(..중략..)

```
for(int i=0;(long unsigned int)i<sizeof(table)/sizeof(table[0]);i++) {
    if(strcmp(cmd_name,table[i].name)==0) {
        if(table[i].function(head,args)) return 1; <- 에러 상태
        return 0; <- 정상 반환
    }
}
printf("Unknown command or permission denied.");
return 1;
```

위와 같이 알 수 있으며,

위 코드에서 if(table...) 부분은 각 함수가 에러가 나면 1을 반환하도록 되어있기에, 만약 1(True)가 return 되면 main.c에게도 1을 반환해 에러가 발생하였음을 알리는 부분이다.

7. 예외 처리 설계

각 명령어에 대한 에러 처리는 명령어 명세에서 명시하였으므로, 명시하지 않은 에러 케이스에 대해서 아래에서 소개하겠음.

```
#ifdef ADMIN_MODE
    #define MODE "admin> "
#elif defined(CLIENT_MODE)
    #define MODE "client> "
#else
    #error "Define either -DADMIN_MODE or -DCLIENT_MODE when compiling."
#endif
```

파일 빌드시에 만약 Flag가 ADMIN/CLIENT Flag가 모두 선언되지 않았을 경우 (컴파일)에러가 발생한다.

```
FILE *fp = fopen(cmd_file, "r");
if (fp == NULL)
{
    printf("Error: failed to load the command file.\n");
    return;
}
```

main.c에서 커맨드 파일을 통해 Mini Shell을 작동시키는 경우, 파일 읽어오기가 실패하면 에러 문구를 출력한 후 즉시 종료한다.

```
if(cmd_process(&head, input) != 0) {
    printf(" Skipped line %d\n",count);
}
```

출력 포맷을 맞추기 위해 각 에러 케이스 발생 시 줄바꿈이 일어나지 않게 해놓았다. Skipped line 문구까지 이어서 출력하기 위한 일종의 꼼수이며, 위 에러문구를 출력한 후에 줄바꿈이 일어나며, 위 count는 커맨드 파일로부터 fgets로 한 줄씩 읽어올때 마다 1씩 증가한다.

커맨드 파일이 아닌 바로 shell로 진입한 경우는 printf("\n); 로 줄바꿈만 일어나도록 되어있다.

(...중략...)

```
if(cmd_name == NULL) {
    printf("Error: Please enter the command.");
    return 1;
}

for(int i=0;(long unsigned int)i<sizeof(table)/sizeof(table[0]);i++) {
    if(strcmp(cmd_name,table[i].name)==0) {
        if(table[i].function(head,args)) return 1;
        return 0;
    }
}

printf("Unknown command or permission denied.");
return 1;
```

command.c에서 매핑 함수로 분기시켜 주는 cmd_process 함수로 명령어 이름 부분이 비어있거나, 함수 테이블에 없는 경우 에러 문구를 띄운다.

```
input[strcspn(input, "\n")] = 0;
    if (input[0] == '\\0' || input[0] == '#') {
        continue;
    }
```

main.c에서 커맨더 파일을 통해 실행하는 경우 줄바꿈 개행문자를 널문자로 변환한 하였으니 널문자 혹은 주석 시작을 알리는 #을 만나는 경우는 커맨더 파일 읽기에서 제외

8. 테스트 케이스 및 결과

필수 점수:	108 / 100 pt
보너스 점수:	5 / 15 pt
어드밴스드:	30 / 30 pt

총 점수:	143 / 145 pt
-------	--------------

=== BONUS CSV-05: sort name + save [PRD §16] (+4pt) ===

[BONUS FAIL] CSV-05: sort name → save → 알파벳 순서 CSV 정답 일치 (0/4pt)

줄 2:

실제 : 3,Charlie,95

정답 : 1,Alice,90

줄 3:

실제 : 1,Alice,90

정답 : 2,Bob,85

줄 4:

실제 : 2,Bob,85

정답 : 3,Charlie,95

=== BONUS CSV-06: sort score + save [PRD §16] (+4pt) ===

[BONUS FAIL] CSV-06: sort score → save → 점수 오름차순 CSV 정답 일치 (0/4pt)

줄 2:

실제 : 1,Alice,90

정답 : 2,Bob,85

줄 3:

실제 : 3,Charlie,95

정답 : 1,Alice,90

줄 4:

실제 : 2,Bob,85

정답 : 3,Charlie,95

=== BONUS TC40 + TC43: sort 메시지 / 에러 [PRD §16] (+2pt) ===

[BONUS FAIL] TC40: sort name → 'sorted by name' 메시지 (0/1pt)

출력: [Admin Program] Loaded 3 students from /tmp/grader_wtp6fsg_/s_sort.csv. admin> Unknown command or permission denied. admin> Goodbye.

[BONUS FAIL] TC43: sort badkey → Error (0/1pt)

출력: [Admin Program] Loaded 3 students from /tmp/grader_wtp6fsg_/s_sort.csv. admin> Unknown command or permission denied. admin> Goodbye.

평가 코드 실행 시, 기본 및 어드밴스드 케이스는 모두 통과하며, 구현해놓지 않은 보너스 코드인 sort 함수의 점수는 얻지 못하는 모습을 확인할 수 있다.

```
leejeongchan@jcleee04:~/CP03_TP/term-project-jcleee2004$ ./admin_shell stu
[Admin Program]
Error: fail to load the file.
admin>
```

또한, 사용자가 파일 이름을 잘못 입력한 경우엔 파일 불러오기를 실패한 후 입력을 계속 입력받아가는 모습을 확인 할 수 있다.

허나, 해당 경우엔 파일 경로를 종료하고 다시 잡아주기 전까지는 데이터를 유효한 위치에 저장할 가능성이 매우 낮다.

9. 한계점 및 개선 방향

한계점

- 현재 sort 함수가 미구현되어 있으며, 데이터 삽입시에 특정한 정렬 기준 없이, 사용자가 입력한 데이터 순서대로 삽입이 되기에 데이터를 한눈에 인식하는데에 있어서 어려움이 생길 수 있다.
- 보너스 기능으로 존재하는 "작업 내용 저장하기 전 종료시 경고"의 구현도 되어있지 않아, 기껏 작업한 내용이 증발하는 경우도 생길 수 있다.

- 초기 파일을 읽어올 때 파일 경로를 잘못 입력한 경우 바로 종료되는 것이 아니라, 사용자가 직접 exit 명령어를 활용해 Mini Shell을 종료시킨후 다시 유효한 경로로 실행시켜야 하는 번거로움이 존재한다.
- 현재 메모리 누수, 댕글링 포인터등의 에러를 확인 할 수 있는 방도가 해당 프로그램에 존재하지 않는다.
- Mini Shell 실행 시 -f 까지만 입력한 경우 에러가 발생하고 Shell이 실행 되기는 하나, 파일 경로를 재지정할 방도가 없기에, 무의미한 실행이다.

개선방안

sort함수 구현

- students.c 내부에 sort 함수를 id,score,name별로 기능하도록 구현하며, 이때 기본 정렬을 id를 통해 이뤄질수 있도록 한다.
- 해당 기능의 구현은 각 노드의 연결을 뒤바꾸는 swap함수를 따로 정의하거나, 노드 간 연결을 그대로 유지하며 내부 필드 값만 바꾸는 두 가지 방법이 있을 듯 하다.
- 값이 유효성이 증명된 경우, id,score는 정수형이니 직접 비교하면 되며, name은 string 헤더의 strcmp 함수를 활용하여 정렬한다.

저장없이 종료시 경고 기능 구현

- main.c에 전역 변수를 하나 만들어, add,update,delete의 작업이 일어난 경우 해당 전역 변수를 1 또는 0(뒤편이었던 기존 값과 반대로)로 업데이트 하도록 한후, exit 명령어 입력시에 해당 값에 따라 경고를 띄운 후 해당 전역 변수 값을 다시 초기값으로 되돌리는 방식으로 코드를 변경한다.

메모리 누수 및 댕글링 포인터 등 메모리 문제 탐지 구현

- Makefile 실행 target으로 -fsanitize=address flag를 추가한 빌드 명령어를 추가한다.

경로 오류시 무의미한 Shell 진입 방지

- shell 진입 함수에서 처음 reload 함수를 호출하고 이때 반환값이 false 즉 에러가 난 경우(리턴 값이 0인 경우), 명령어를 계속 이어 받는게 아니라 즉시 종료하도록 return 문을 추가한다.